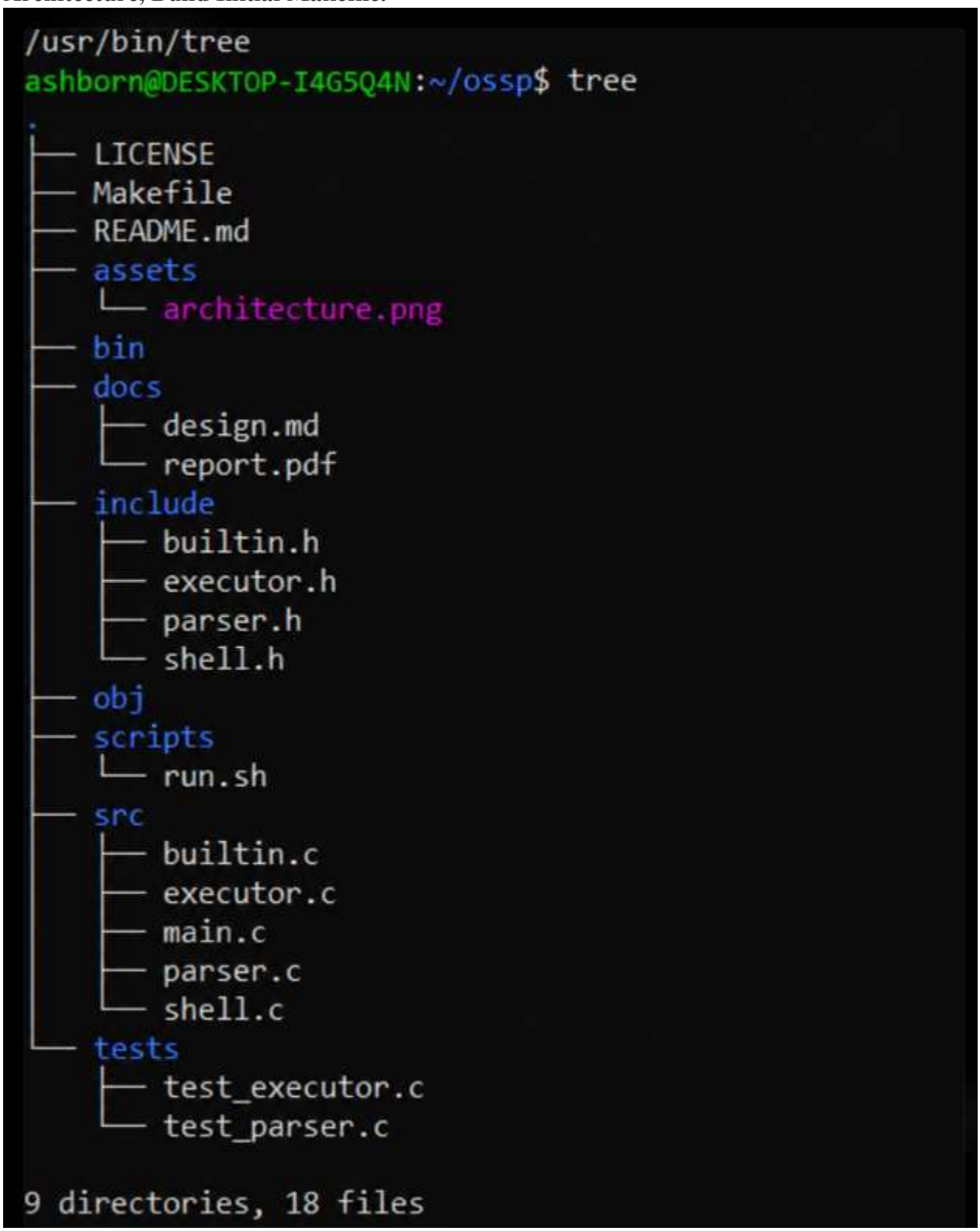
WEEK-1 SKILL

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.



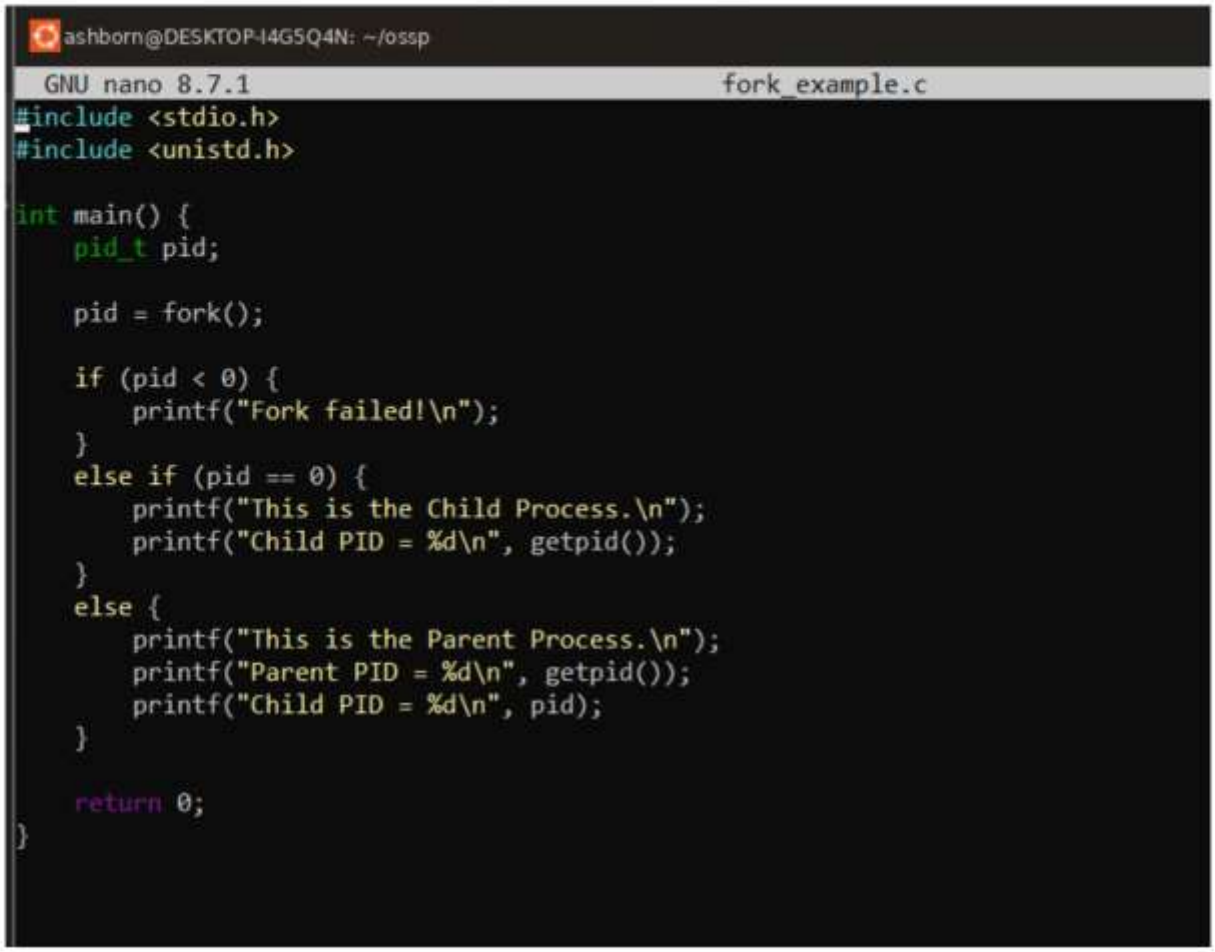

ashborn@DESKTOP-I4G5Q4N: ~/ossp
GNU nano 8.7.1 fork_example.c *
#include <stdio.h>
#include <unistd.h>
int main() {
pid_t pid;
pid = fork();
if (pid < 0) {
printf("Fork failed!\n");
}
else if (pid == 0) {
printf("This is the Child Process.\n");
printf("Child PID = %d\n", getpid());
}
else {
printf("This is the Parent Process.\n");
printf("Parent PID = %d\n", getpid());
printf("Child PID = %d\n", pid);
}
return 0;
}

OUTPUT:

```
ashborn@DESKTOP-I4G5Q4N:~/ossp$ nano fork_example.c
ashborn@DESKTOP-I4G5Q4N:~/ossp$ gcc fork_example.c -o fork_example
ashborn@DESKTOP-I4G5Q4N:~/ossp$ ./fork_example
This is the Parent Process.
Parent PID = 1568
Child PID = 1569
This is the Child Process.
Child PID = 1569
ashborn@DESKTOP-I4G5Q4N:~/ossp$
```

REPORT: I successfully installed the Linux environment and configured the GCC compiler. I created a Git repository and organized the required project folders and files. I also understood the basic shell architecture and created a simple Make file to build the project.

1. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**


ashborn@DESKTOP-I4G5Q4N: ~/ossp
GNU nano 8.7.1 fork_example.c
#include <stdio.h>
#include <unistd.h>
int main() {
pid_t pid;
pid = fork();
if (pid < 0) {
printf("Fork failed!\n");
}
else if (pid == 0) {
printf("This is the Child Process.\n");
printf("Child PID = %d\n", getpid());
}
else {
printf("This is the Parent Process.\n");
printf("Parent PID = %d\n", getpid());
printf("Child PID = %d\n", pid);
}
return 0;
}

OUTPUT:

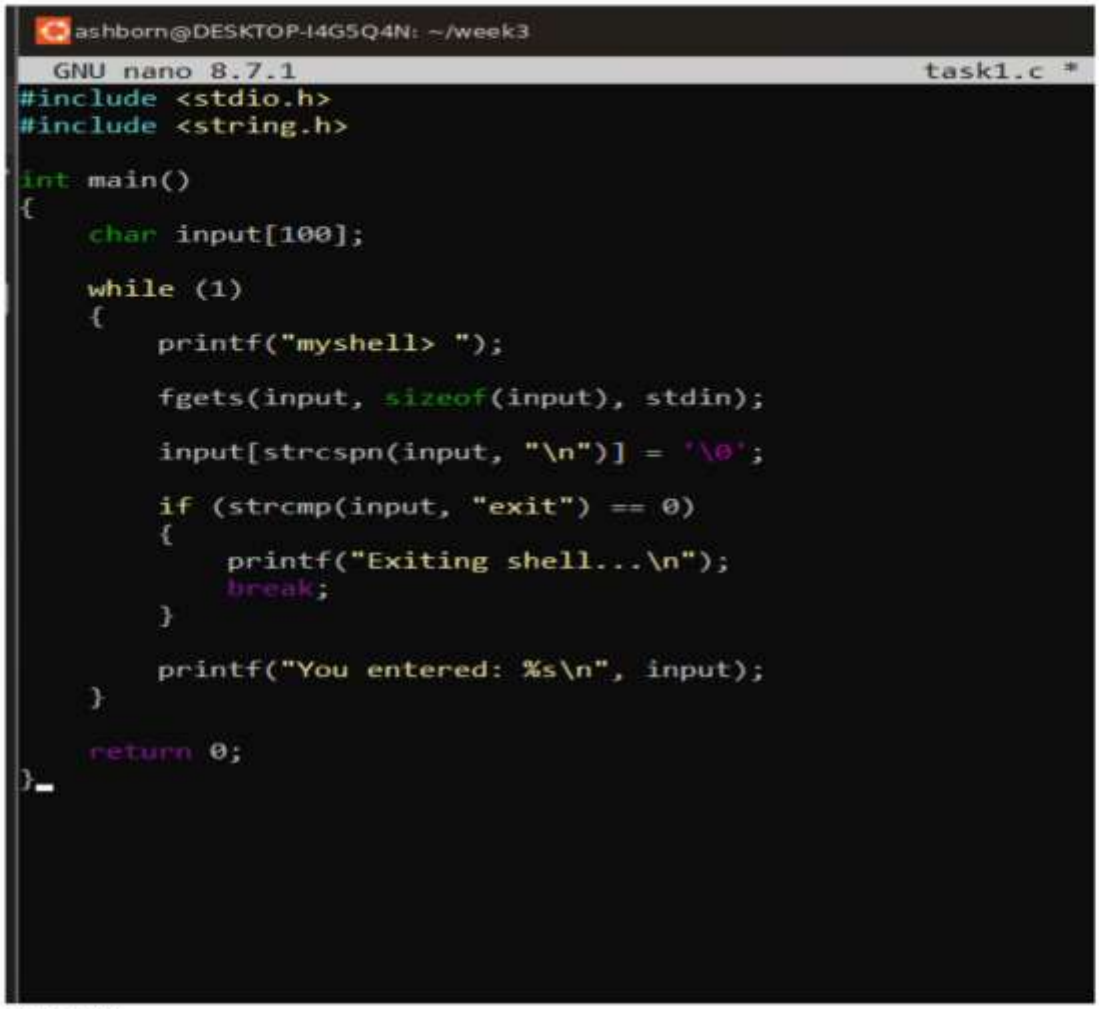
```
ashborn@DESKTOP-I4G5Q4N:~/ossp$ nano fork_exec.c
ashborn@DESKTOP-I4G5Q4N:~/ossp$ gcc fork_exec.c -o fork_exec
ashborn@DESKTOP-I4G5Q4N:~/ossp$ ./fork_exec
Parent Process
Child Process
total 72
-rw-r--r-- 1 ashborn ashborn     0 Aug  4 05:07 LICENSE
-rw-r--r-- 1 ashborn ashborn     0 Aug  4 05:07 Makefile
-rw-r--r-- 1 ashborn ashborn     0 Aug  4 05:07 README.md
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:09 assets
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:07 bin
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:08 docs
-rwxr-xr-x 1 ashborn ashborn 16088 Aug  4 05:31 fork_example
-rw-r--r-- 1 ashborn ashborn   443 Aug  4 05:31 fork_example.c
-rwxr-xr-x 1 ashborn ashborn 16040 Aug  4 05:34 fork_exec
-rw-r--r-- 1 ashborn ashborn   389 Aug  4 05:34 fork_exec.c
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:08 include
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:07 obj
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:08 scripts
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:07 src
drwxr-xr-x 2 ashborn ashborn  4096 Aug  4 05:08 tests
```

REPORT: I executed the fork() program and observed how a parent process creates a child process. I understood the working of the exec() family and learned how processes are replaced during execution. I also examined the process tree and practiced tracing system calls to understand how a program interacts with the operating system.

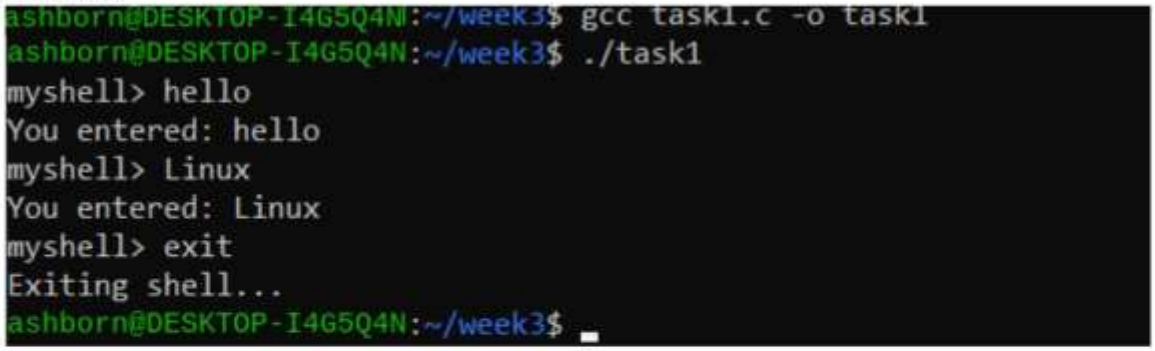
SESSION -2

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
ashborn@DESKTOP-I4G5Q4N: ~/week3
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week3
ashborn@DESKTOP-I4G5Q4N:~$ cd week3
ashborn@DESKTOP-I4G5Q4N:~/week3$ pwd
/home/ashborn/week3
```



OUTPUT:



REPORT: I created a program to copy the contents of File1.txt into File2.txt using the open(), read(), write(), and close() system calls. I successfully executed the program and verified that the contents were copied correctly. I understood how files are opened, read, written, and closed using system calls.

TASK-2: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

CODE:

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>

#define MAX_INPUT 100

int main()
{
    char input[MAX_INPUT];
    int index = 0;
    char ch;

    struct termios oldt, newt;

    // Save current terminal settings
    tcgetattr(STDIN_FILENO, &oldt);

    // Disable canonical mode and echo
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("my_shell> ");
    fflush(stdout);

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        // Enter key
        if (ch == '\n' || ch == '\r')
        {
            input[index] = '\0';

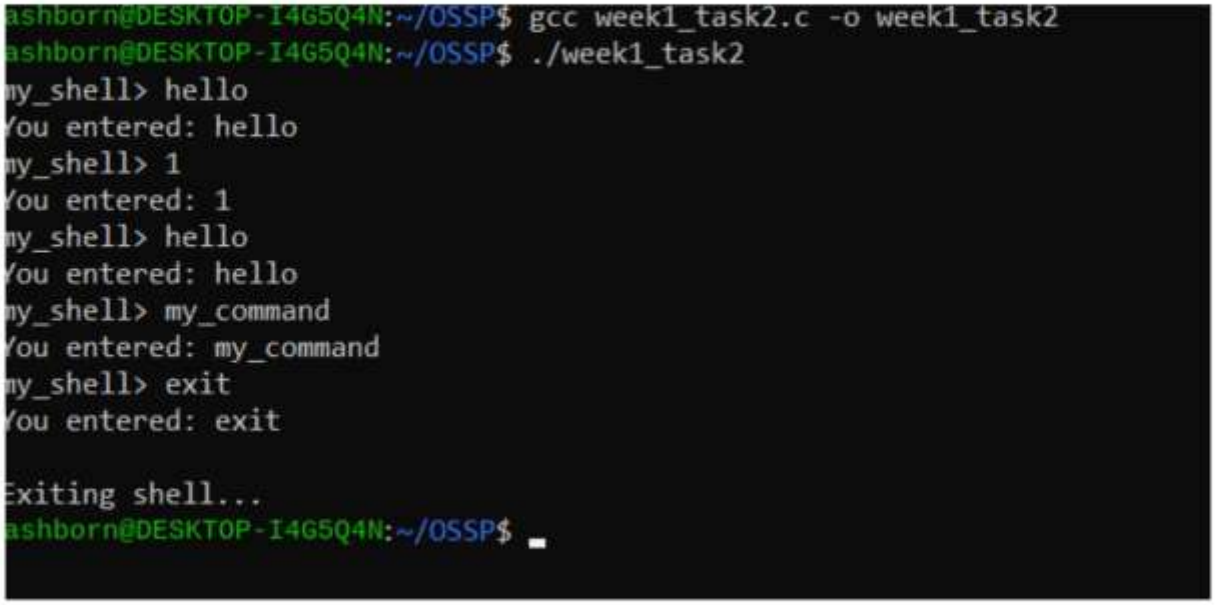
            printf("\nYou entered: %s\n", input);

            if (strcmp(input, "exit") == 0)
            {
                break;
            }

            index = 0;

            printf("my_shell> ");
            fflush(stdout);
        }
```

OUTPUT:



REPORT: I implemented keyboard input handling for the shell and tested different types of user input. I handled the Backspace key to remove characters and used the Enter key to process the command. I also tested multi-character commands and understood how the input buffer stores and manages the characters entered by the user.